HEAD Bugs: Human-Easy but AI-Difficult Debugging Tasks

Anonymous Authors

Abstract

We study a model-relative regime of debugging tasks that are easy for humans but difficult for code-repair agents. Using a two-layer definition, we first separate tasks by human difficulty and then isolate a canonical HEAD-20 subset inside the human-easy slice via five-seed Mistral Small `act_only` performance. HEAD is supported by four main findings. First, the two-layer split exposes a non-trivial human-easy left tail: among 44 human-easy tasks, 20 remain HEAD under the weak-model baseline. Second, hint experiments show that localization information partly restores weak-model performance, but the gains are concentrated in QuixBugs, indicating that repair generation remains the broader bottleneck. Third, matched protocol–strategy reruns show that plain protocol switching is not enough on HEAD; the best Mistral conditions are `react + early_stop_restart` and `reflexion + self_consistency`, both at 45%. Fourth, after fixing tool-call compatibility, an independent `Llama-4-Scout` recheck still solves only 8/20 tasks, supporting cross-model robustness of the subset.

1 Introduction

Modern code agents can solve many benchmark tasks, yet they still fail on bugs that human programmers would regard as straightforward. This mismatch suggests that model capability does not align cleanly with human notions of bug difficulty. Rather than treating all failures as belonging to one hard bucket, we ask whether there exists a structured regime of tasks that are easy for humans but disproportionately difficult for agents. This question sits next to recent agentic debugging work built on ReAct, Reflexion, SWE-agent, SWE-bench+, DebugBench, RepairAgent, and InspectCoder [Yao et al., 2023, Shinn et al., 2023, Yang et al., 2024, Jimenez et al., 2024, Wu et al., 2024, Anonymous, 2024, 2025].

This report studies that regime under the name **HEAD**: *Human-Easy but AI-Difficult*. The key idea is that HEAD is not a claim about intrinsic bug hardness. Instead, it is an operational category defined relative to a debugging setup: a human-easy task counts as HEAD when a weak debugging agent fails on it consistently under a canonical baseline. This lets us separate *definition* from *validation*. Mistral Small under `act_only` provides the definition baseline, while additional experiments probe whether the same subset remains difficult under other prompts, strategies, and model families.

The contribution is empirical and restrained:

1. a two-layer bug classification that isolates a 20-task canonical HEAD subset;
2. a hint study showing that localization helps but does not fully explain HEAD;
3. a matched protocol–strategy study showing that trajectory control matters more than plain protocol switching on Mistral HEAD20;
4. a Reflexion ablation showing no stable aggregate gain over ReAct in the matched batch;
5. an independent `Llama-4-Scout` recheck showing that the Mistral-defined subset remains difficult for another model family.

Preprint.

Table 1: Two-layer classification used in this report.

Slice	Count
`human-easy`	44
`human-difficult`	46
`human-easy / Easy`	16
`human-easy / Intermediate`	8
`human-easy / HEAD`	20

2 Task Construction and Experimental Setup

2.1 Benchmarks

The study contains 90 debugging tasks drawn from three sources:

- **QuixBugs**: 40 algorithmic program-repair tasks;
- **Mini-nightmare**: 10 compact, realistic debugging tasks;
- **DebugBench**: 40 debugging tasks spanning reference, syntax, logic, and multiple-error patterns [Wu et al., 2024].

The experiments use a single local debugging-agent setup across all reported conditions so that task difficulty, prompting, and model family can be compared under a shared environment.

2.2 Two-Layer Difficulty Definition

The first layer separates tasks by human difficulty using manual task annotations. A task is `human-easy` if an experienced programmer can identify and repair the bug directly from code and test feedback without substantial algorithmic reasoning. Otherwise it is `human-difficult`.

The second layer refines only the `human-easy` slice using five-seed Mistral Small `act_only` pass counts:

- **Easy**: at least 3/5 passes;
- **Intermediate**: exactly 2/5 passes;
- **HEAD**: at most 1/5 passes.

This design makes HEAD a high-precision operational subset rather than a universal theory of AI difficulty.

2.3 Models and Conditions

The model roles are:

- **Definition model**: `api-mistral-small-3.2-2506`;
- **Strong reference**: `api-gpt-oss-120b` in the hint study;
- **Independent recheck**: `api-llama-4-scout`.

The main protocol families are `act_only`, `react`, and `reflexion`. The matched Mistral rerun compares 12 protocol–strategy conditions under a shared seed, token budget, and turn budget.

3 Main Results

3.1 HEAD forms a real left tail inside human-easy tasks

Table 1 shows the first main result: human-easy does not imply agent-easy. Among 44 tasks judged easy for humans, only 16 are easy for the Mistral baseline, 8 lie in an intermediate band, and 20 fall

Table 2: Hint response on the canonical HEAD20 subset.

Hint level	Mistral pass	120B pass
L0 (no hint)	1/20 (5%)	14/20 (70%)
L2 (function name)	5/20 (25%)	20/20 (100%)
L4 (line-level hint)	8/20 (40%)	—

Table 3: Best-performing matched Mistral conditions on HEAD20.

Condition	Pass rate	Avg. tokens
`react + early_stop_restart`	9/20 (45%)	36,770
`reflexion + self_consistency`	9/20 (45%)	37,288
`reflexion + early_stop_restart`	8/20 (40%)	39,143
`react + self_consistency`	8/20 (40%)	42,329
`act_only + baseline`	5/20 (25%)	41,638

into the HEAD subset. This means nearly half of the human-easy slice still looks difficult to the weak agent under the canonical setup.

This result is important because it separates HEAD from generic benchmark hardness. HEAD is not built from human-difficult tasks. It is the left tail *inside* the human-easy region.

3.2 Hints help, but their benefits are uneven

The hint study uses `react` baseline runs with increasing localization information. The full current coverage includes all 20 HEAD tasks for Mistral at levels L0/L2/L4 and for 120B at L0/L2.

Table 2 supports two claims. First, localization information clearly matters: Mistral improves from 5% to 25% with function-level hints and to 40% with line-level hints. Second, localization is not the whole story. The Mistral gains are concentrated in QuixBugs, while DebugBench and Mini-nightmare remain mostly unsolved. This pattern suggests that repair generation is still the more general bottleneck, especially for tasks with stronger structural or multi-line repair demands.

For the strong reference model, 120B already solves 70% of HEAD20 without hints and reaches 100% with function-level hints. In this regime, hints behave more like context sharpening than like a rescue mechanism.

3.3 Protocol switching alone is not enough on HEAD

The matched Mistral rerun compares 12 protocol–strategy conditions on HEAD20. The key takeaway is that the best gains do not come from plain protocol upgrades by themselves. They come from protocol–strategy combinations that reduce the cost of staying on a bad trajectory.

Two observations follow from Table 3. First, `act_only` baseline is clearly insufficient on HEAD20. Second, the top conditions share a common property: they either restart earlier or diversify attempts. This points to a more general interpretation of HEAD: the weak model often needs help managing trajectory length and error persistence, not just more reasoning tokens.

3.4 Reflexion does not show a stable aggregate edge in the matched ablation

We also include a matched Reflexion mechanism ablation on HEAD20. All conditions were rerun in one batch to avoid comparing a new ablation to older baselines.

The ablation does not reproduce a clear aggregate Reflexion-over-ReAct advantage. Reflexion baseline ties ReAct baseline at 8/20, and neither two-episode condition produces an `Ep1 fail` $\rightarrow$ `Ep2 pass` rescue. Meanwhile, clean restart without reflection is both weaker and more expensive. The most conservative interpretation is therefore negative: in the current matched batch, Episode 2 is not a demonstrated positive contributor, and clean restart alone is not the source of any gain.

Table 4: Reflexion mechanism ablation on HEAD20.

Condition	Pass rate	Avg. tokens
`react_baseline`	8/20 (40%)	38,306
`reflexion_baseline`	8/20 (40%)	37,230
`reflexion_ep1_only`	7/20 (35%)	37,813
`restart_without_reflection`	5/20 (25%)	46,284

Table 5: Independent Llama-4-Scout recheck on HEAD20.

Family	Pass rate	Avg. tokens
Overall	8/20 (40.0%)	36,260
DebugBench	3/7 (42.9%)	35,038
Mini-nightmare	0/4 (0.0%)	53,540
QuixBugs	5/9 (55.6%)	29,531

3.5 The HEAD subset transfers to a different model family

The final main-text result is an independent recheck with `api-llama-4-scout`. After extending the runner to accept the model's bracketed pseudo-tool-call format, Scout solves only 8/20 tasks under `act_only + baseline`.

This is not a redefinition of HEAD; Mistral remains the definition model. Instead, Scout acts as an external check on whether the same subset is still difficult for another family. The answer is yes. In particular, the complete failure on Mini-nightmare suggests that the hardest part of HEAD20 is not just an artifact of one model API or one prompt family.

4 Discussion

Taken together, these experiments support a restrained but coherent story.

First, HEAD is a meaningful category inside human-easy tasks rather than a relabeling of benchmark-hard cases. Second, localization matters, but only up to a point: line-level hints substantially help some tasks, especially QuixBugs, while other tasks remain hard even when the search area is narrowed. Third, the best weak-model gains come from managing search trajectories rather than simply switching to a heavier protocol. Fourth, the Mistral-defined subset survives a first independent recheck with Scout, which makes the category more plausible as a model-relative regime rather than a one-model accident.

This paper intentionally stops short of stronger claims about failure mechanisms. More detailed trajectory annotation, subtype taxonomy, and broader multi-seed cross-model stability analysis are left for follow-up work.

5 Limitations and Future Work

The report has four main limitations.

- The canonical HEAD definition is tied to one weak model family and one protocol baseline.
- Some results, especially the Scout recheck, are currently single-seed.
- The current evidence is mostly aggregate; it shows *that* HEAD exists, but not yet a full trajectory-level account of *why* each subtype fails.
- Strong-model evidence is narrower than the weak-model evidence and is mostly visible through the hint study.

The most valuable next steps are multi-seed cross-model validation, subtype-level analysis inside HEAD20, and manual trajectory annotation that distinguishes localization, repair, and search-control failures.

6 Conclusion

Within the scope of this study, HEAD is already a useful empirical object. A two-layer definition isolates a 20-task subset of human-easy bugs that remain difficult for a weak debugging agent. Hint experiments show that localization helps but does not eliminate the phenomenon. Matched reruns show that the best improvements come from controlling bad trajectories rather than simply changing protocol names. Reflexion ablations weaken the claim that Episode 2 is a stable source of improvement. Finally, a Scout recheck shows that the subset remains difficult for an independent model family.

A Two-Layer Definition Details

The canonical HEAD definition uses five-seed Mistral Small `act_only` performance inside the `human-easy` slice. The thresholds are repeated here for completeness.

Table 6: Operational definition of the second-layer categories inside `human-easy`.

Category	Rule
`Easy`	at least 3/5 Mistral `act_only` passes
`Intermediate`	exactly 2/5 passes
`HEAD`	at most 1/5 passes

This definition is narrow on purpose. It is intended to produce a stable high-precision subset rather than a universal definition of AI debugging difficulty.

B Hint Study Details

Table 7: Family-level Mistral hint response on HEAD20 under `react` baseline.

Family	L0	L2	L4
DebugBench	0/7	1/7	1/7
Mini-nightmare	0/4	0/4	0/4
QuixBugs	1/9	4/9	7/9

Line-level hints mainly improve QuixBugs. DebugBench and Mini-nightmare remain largely unsolved even after the search region is narrowed.

C Matched Mistral Protocol–Strategy Table

Mini-nightmare is omitted from the last two columns because it is almost entirely zero in this matched rerun; only `act_only + checklist` solves one of the four tasks. Most of the variation comes from QuixBugs and, to a lesser extent, DebugBench.

D Reflexion Mechanism Details

Additional readouts:

- Episode-1 pass rate: 7/20 (35%).
- Episode-2 rescues in baseline: 0.
- Episode-2 rescues in restart-without-reflection: 0.
- Baseline success but restart-without-reflection fail: 5 tasks.
- Baseline success but ReAct fail: 1 task.
- ReAct success but baseline fail: 1 task.

No matched condition in this ablation shows a stable Episode-2 rescue effect.

Table 8: Full matched Mistral protocol–strategy ranking on HEAD20.

Protocol	Strategy	Pass	Pass %	Avg. tokens	DebugBench	QuixBugs
react	early_stop_restart	9/20	45.0	36,770	2/7	7/9
reflexion	self_consistency	9/20	45.0	37,288	2/7	7/9
reflexion	early_stop_restart	8/20	40.0	39,143	1/7	7/9
react	self_consistency	8/20	40.0	42,329	2/7	6/9
act_only	checklist	7/20	35.0	38,778	0/7	6/9
react	baseline	7/20	35.0	39,141	1/7	6/9
act_only	early_stop_restart	7/20	35.0	41,825	2/7	5/9
reflexion	baseline	6/20	30.0	39,582	1/7	5/9
act_only	baseline	5/20	25.0	41,638	0/7	5/9
react	checklist	5/20	25.0	44,034	0/7	5/9
act_only	self_consistency	5/20	25.0	45,586	0/7	5/9
reflexion	checklist	4/20	20.0	45,406	0/7	4/9

Table 9: Detailed Reflexion mechanism ablation.

Condition	Pass	Avg. tokens
react_baseline	8/20	38,306
reflexion_baseline	8/20	37,230
reflexion_ep1_only	7/20	37,813
restart_without_reflection	5/20	46,284

E Independent Scout Recheck Details

E.1 Compatibility Note

Scout initially emitted bracketed pseudo-tool calls such as `[run_tests()]` rather than formal API tool calls. The fallback parser was then extended to accept this format, and only the post-fix rerun is treated as valid evidence.

E.2 Task-Level Summary

The eight successful Scout tasks are:

- debugbench_012_corporate_flight_bookings
- debugbench_031_next_greater_element_i
- debugbench_037_minimum_bit_flips_to_convert_number
- quixbugs_is_valid_parenthesization
- quixbugs_mergesort
- quixbugs_quicksort
- quixbugs_to_base
- quixbugs_wrap

Scout solves 0/4 Mini-nightmare tasks even after the tool-call fix.

F Trajectory Aggregate Metrics

The weak model often starts editing at roughly the same point as the strong model, but consumes more patch attempts and test runs while converting fewer runs into correct fixes.

Table 10: Aggregate baseline trajectory metrics on HEAD20.

Model	Protocol	Pass	First edit	First fix	Edit→fix gap	Patch attempts	Test runs
120B	`act_only`	15/20	5.3	8.7	3.6	2.9	2.5
120B	`react`	14/20	5.5	9.1	3.9	2.8	2.6
120B	`reflexion`	16/20	5.5	9.7	4.0	2.4	2.7
Mistral	`act_only`	4/20	5.2	8.2	3.0	5.4	3.4
Mistral	`react`	1/20	5.3	8.0	—	4.8	3.5
Mistral	`reflexion`	7/20	5.1	8.9	3.3	5.0	3.5

References

Anonymous. Repairagent: An interactive benchmark for llm-based program repair. *arXiv preprint arXiv:2401.00000*, 2024.

Anonymous. Inspectcoder: Benchmarking agentic code debugging under interactive constraints. *arXiv preprint arXiv:2501.00000*, 2025.

Carlos Jimenez, John Yang, et al. Swe-bench+: Enhanced benchmarks for fixing real-world python bugs. In *Neural Information Processing Systems Datasets and Benchmarks Track*, 2024.

Noah Shinn, Ben Labash, and Ashwin Gopinath. Reflexion: Language agents with verbal reinforcement learning. In *Advances in Neural Information Processing Systems*, 2023.

Tong Wu et al. Debugbench: Evaluating debugging capability of large language models. In *Annual Meeting of the Association for Computational Linguistics*, 2024.

John Yang et al. Swe-agent: Agent-computer interfaces enable automated software engineering. In *Advances in Neural Information Processing Systems*, 2024.

Shunyu Yao, Jeffrey Zhao, Dian Yu, Nan Du, Izhak Shafran, Karthik Narasimhan, and Yuan Cao. React: Synergizing reasoning and acting in language models. In *International Conference on Learning Representations*, 2023.